

Sistemi Operativi

Luca Spalazzi

l.spalazzi@staff.univpm.it

www.univpm.it/luca.spalazzi



Cos'è l'informatica?

- Computer science is no more about computers than astronomy is about telescopes, biology is about microscopes or chemistry is about beakers and test tubes. Science is not about tools, it is about how we use them and what we find out when we do.

Michael R. Fellows and Ian Parberry

Programma del Corso

U1 – Introduzione

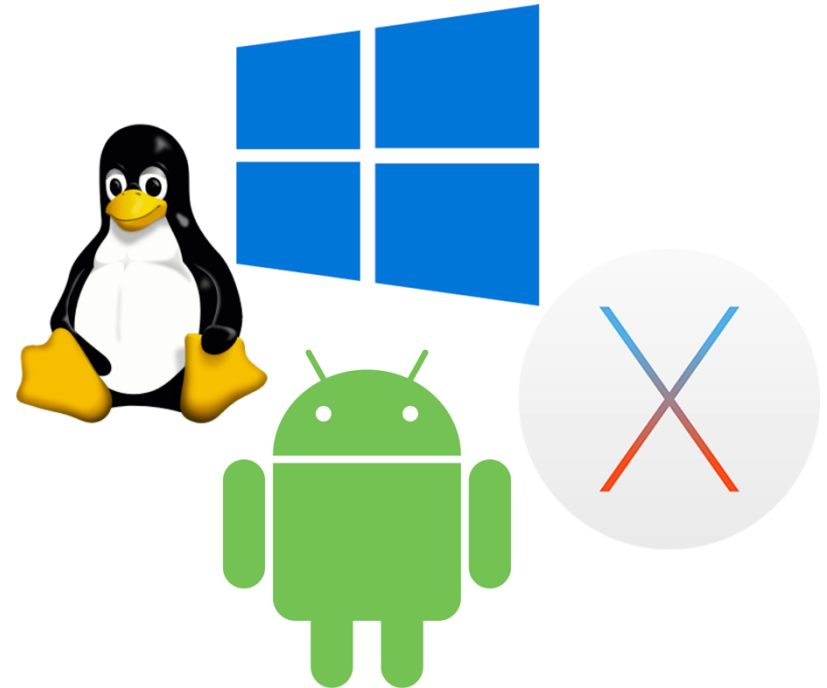
U2 – Processi Concorrenti

U3 – Scheduling Processi

U4 – Memoria

U5 – File System e I/O

U6 – Linux



Sistemi Operativi



Testi di riferimento e materiale didattico

- A. Silberschatz, P. B. Galvin, G. Gagne, Sistemi Operativi, 10^a Edizione, Pearson, 2019
- Copia dei lucidi proiettati a lezione
- Esercizi e approfondimenti proposti dal docente (sito del corso su piattaforma Moodle)

Modalità di esame



Prima prova:

scritta (9 domande) vale 11 punti

– Iscrizione su Esse3Web – **Sistemi Operativi - Scritto**

Voto Prova Scritta	Ammissione all'orale	Quando
voto $\geq 6/11$	ammesso	in un qualsiasi altro appello
voto = 5/11	ammesso	stesso appello della prova scritta
< 5/11	non ammesso	



Seconda prova:

orale (2 argomenti) vale 21 punti

– Iscrizione su Esse3Web – **Sistemi Operativi - Orale**

- Voto finale
 - Somma dei voti presi nelle due prove



<https://learn.univpm.it/course/view.php?id=16349>

- Accesso al sito utilizzando le credenziali di Ateneo
- Il sito serve per
 - Scaricare materiale didattico

e-univpm

- **Luca Spalazzi:**
 - Forum News (sito Moodle)
 - Annunci e news del docente agli studenti
 - Forum Partecipato (sito Moodle)
 - Forum aperto alla partecipazione di tutti gli studenti
 - Ricevimento
 - su appuntamento
 - Recapito di posta elettronica
 - `l.spalazzi@univpm.it`



CYBERSECURITY

Cybersecurity According to JTFCE

Knowledge Areas

Data Security

Software Security

Component Security

Connection Security

System Security

Human security

Organizational Security

Societal Security



Association for
Computing Machinery



ASSOCIATION FOR
INFORMATION SYSTEMS



*Joint Task Force on
Cybersecurity Education*



Cybersecurity @ Univpm

Knowledge Areas	Software Security & Blockchain	Crittografia & Blockchain	Cybersecurity for Networks	Advanced Cybersecurity for IT	Ethical Hacking
<i>Data Security</i>					
<i>Software Security</i>					
<i>Component Security</i>					
<i>Connection Security</i>					
<i>System Security</i>					
<i>Human security</i>					
<i>Organizational Security</i>					
<i>Societal Security</i>					

Laurea Magistrale in Ingegneria Informatica e dell'Automazione

■ 1° anno

- *Project Management per l'ICT*
- Due a scelta tra:
 - *Programmazione Avanzata*
 - Big Data Analytics e Machine Learning
 - Intelligenza Artificiale
- *Software Cybersecurity*
- *Cybersecurity for Networks* o *Ethical Hacking*

■ 2° anno

- Uno a scelta tra:
 - Linguaggi di Programmazione
 - Sistemi Operativi Dedicati
- *Advanced Cybersecurity for IT*
- Due a scelta tra:
 - Computer Graphics e Multimedia
 - Data Science
 - New Generation Databases

Argomenti di tesi (sia interne che aziendali, sia in Italia sia all'estero)

- Progettazione e validazione di software mediante metodi formali
- Progettazione e realizzazione di sistemi basati su blockchain
- Analisi e rilevamento di malware mediante machine learning
- Monitoraggio di sistemi basati su addestramento automatico
- Valutazione del rischio



Cybersecurity @ Univpm



Cybersecurity National Lab



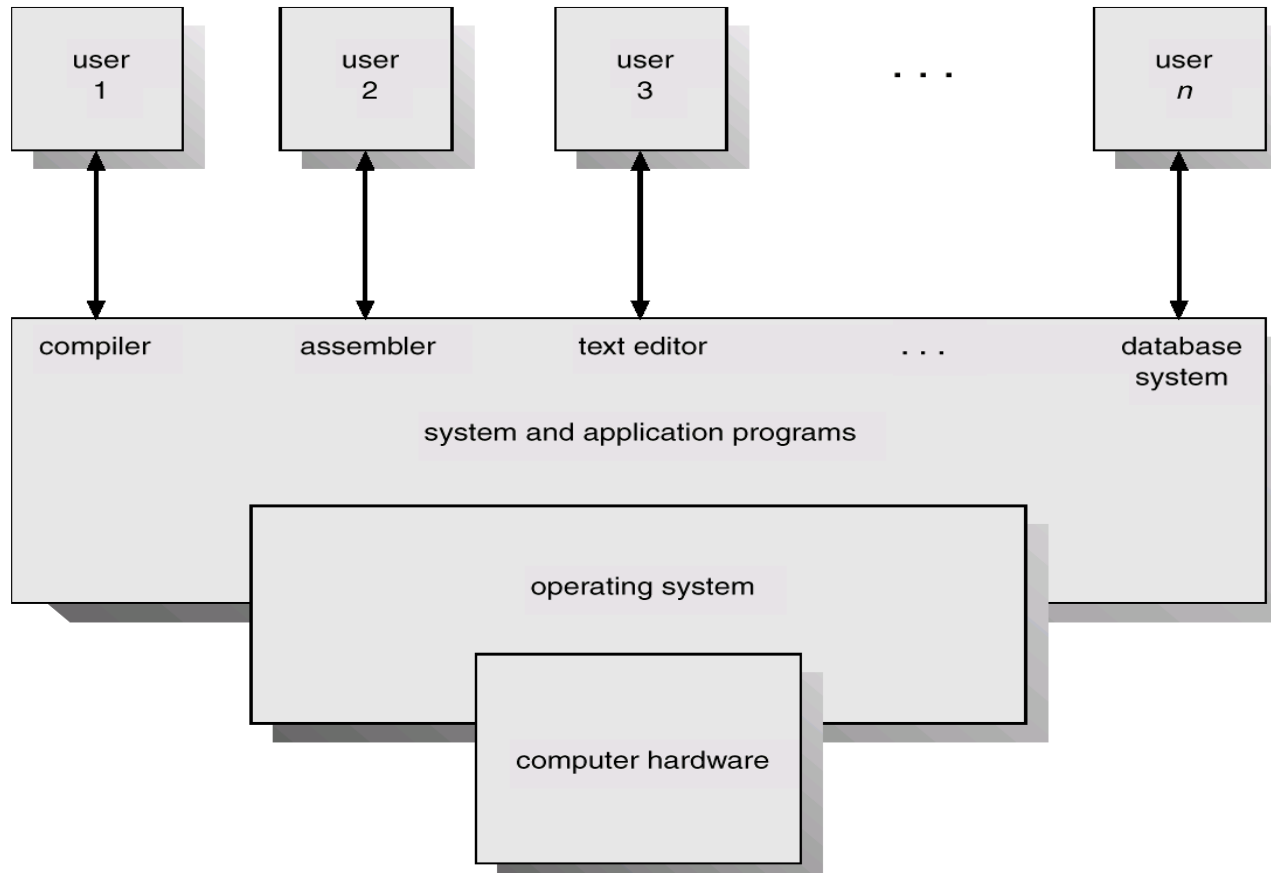
INTRODUZIONE

- Sistemi di Elaborazione
 - Classificazione basata sulle architetture hardware (di Flynn)
 - Classificazione basata sui sistemi operativi
- Struttura di un Sistema Operativo
 - Funzionalità di un SO
 - Servizi di un SO
 - Architetture
 - Monolitica
 - Stratificata
 - Microkernel
 - Modulare
 - Macchina Virtuale



Insieme di risorse hardware e software per
l'elaborazione automatica delle informazioni.

Architettura di un Sistema di Elaborazione

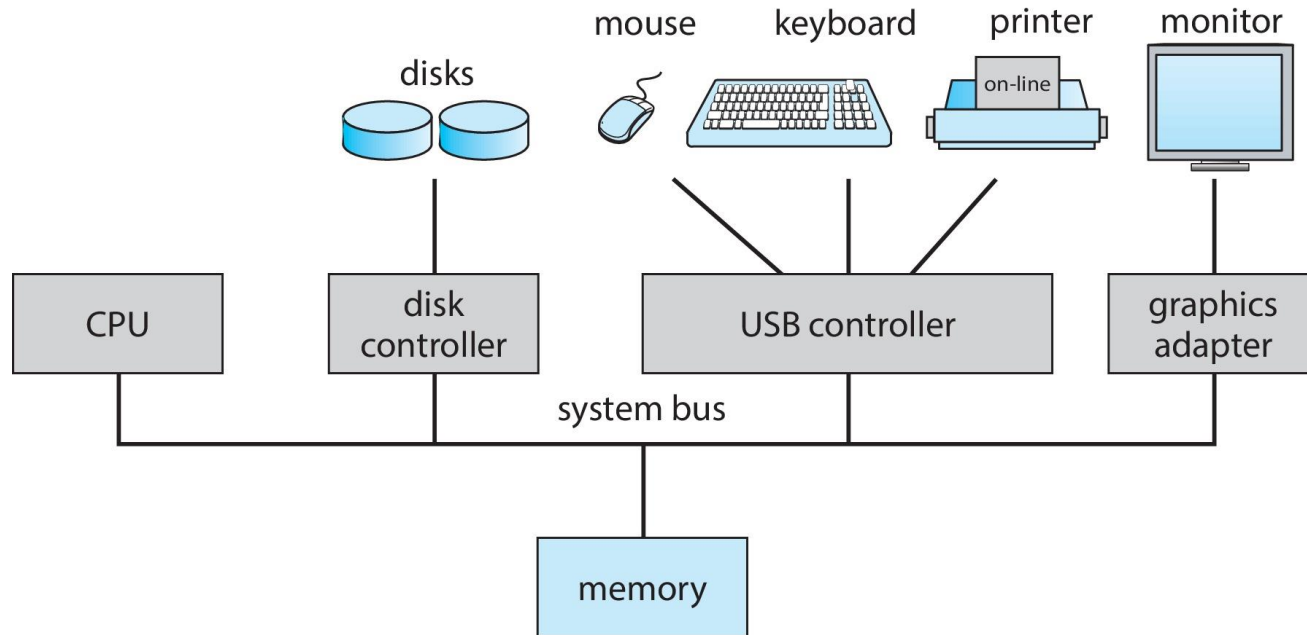


1. **Hardware:**
le risorse di calcolo fondamentali (CPU, memoria, dispositivi di I/O).
2. **Sistema Operativo:**
controlla e coordina l' utilizzo dell' hardware da parte dei vari programmi applicativi di utenti diversi.
3. **Programmi Applicativi:**
definiscono il modo in cui le risorse del sistema devono essere usate per risolvere i problemi di elaborazione dei diversi utenti (es. compilatori, basi di dati, video game, videoscrittura, programmi scientifici, programmi contabili).
4. **Utenti:**
coloro che utilizzano i programmi applicativi (persone, macchine, altri computer).

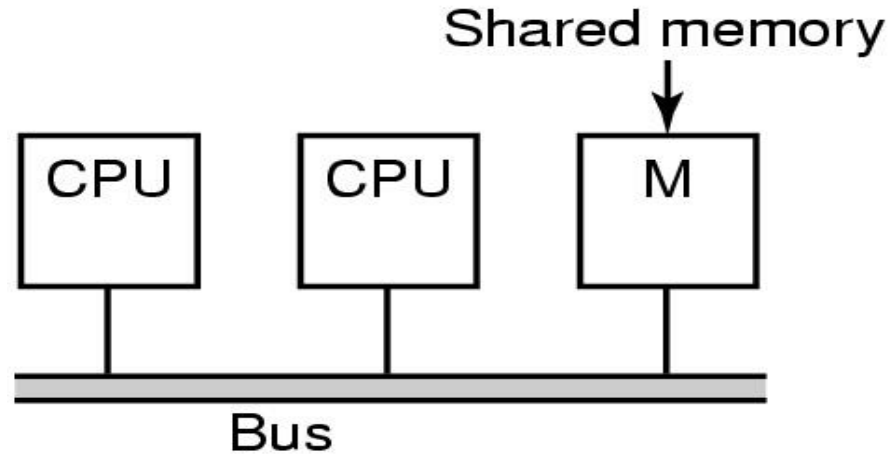
- **CPU o processore** – dispositivo che esegue le istruzioni di un programma per l'elaborazione delle informazioni.
- **Memoria** – insieme di dispositivi per la memorizzazione temporanea (memoria centrale) o permanente (memoria di massa) delle informazioni.
- **Dispositivi di Input/Output** – insieme di dispositivi per l'acquisizione o la restituzione di informazioni.

- **SISD** – Singolo flusso di Istruzioni
– Singolo flusso di Dati
> *tradizionali calcolatori monoprocesso*
- **SIMD** – Singolo flusso di Istruzioni
– Molteplici flussi di Dati
> *ad es. gli array processor*
- **MISD** – Molteplici flussi di Istruzioni
– Singolo flusso di Dati
> *nessun calcolatore noto rientra in questa categoria*
- **MIMD** – Molteplici flussi di Istruzioni
– Molteplici flussi di Dati
> *multiprocessori*
> *multicalcolatori strettamente accoppiati*
> *multicalcolatori lascamente accoppiati*

SISD - Monoprocessori



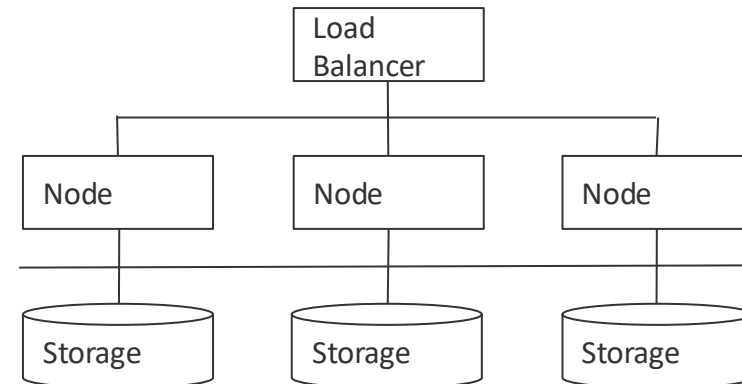
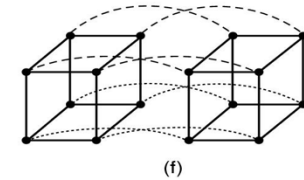
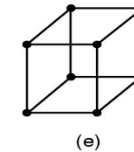
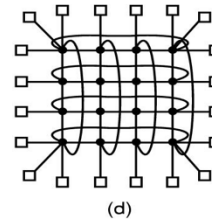
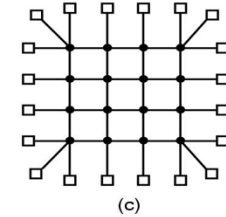
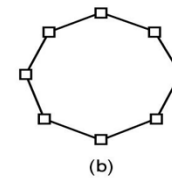
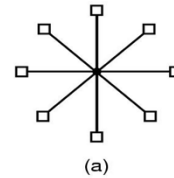
MIMD - Multiprocessori



MIMD - Multicalcolatori strettamente accoppiati

■ Diverse topologie per la rete di interconnessione

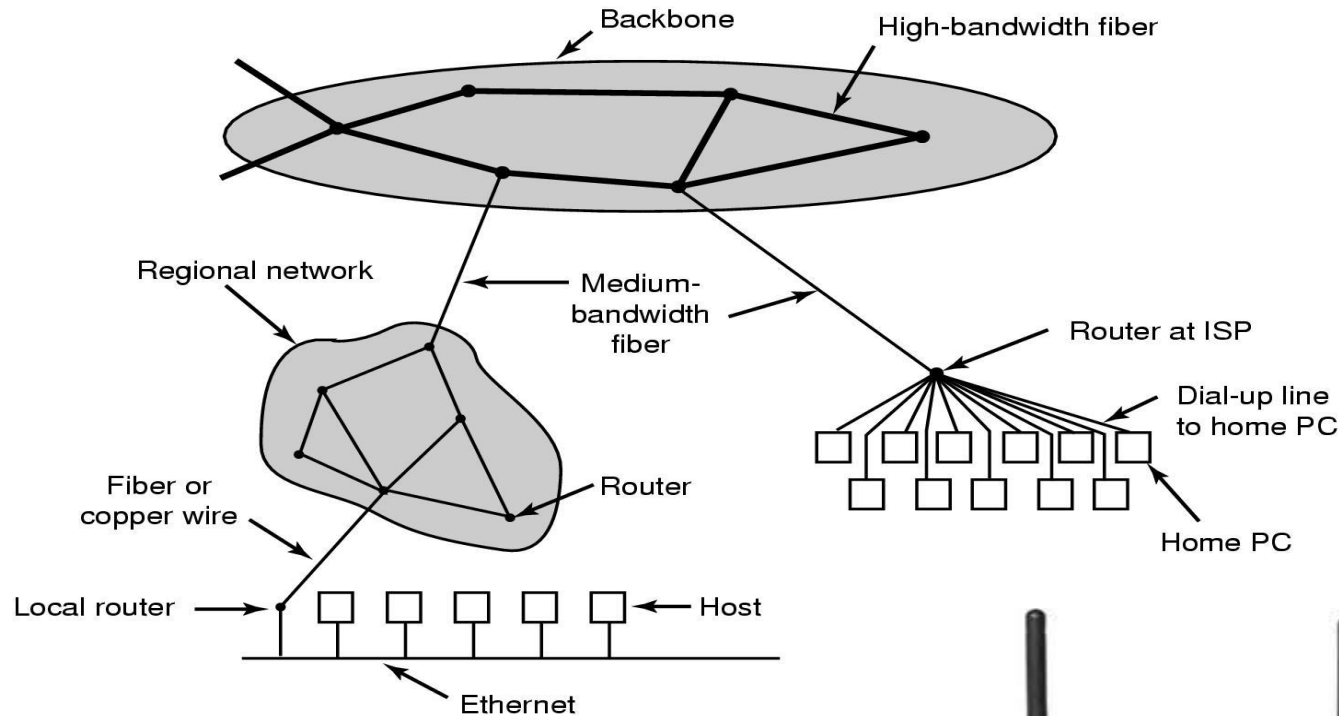
- 1) stella (switch singolo)
- 2) anello
- 3) griglia (grid)
- 4) doppio toro (griglia richiusa)
- 5) cubo
- 6) ipercubo



MIMD - Multicalcolatori lascamente accoppiati

Sistemi Distribuiti

WAN



LAN



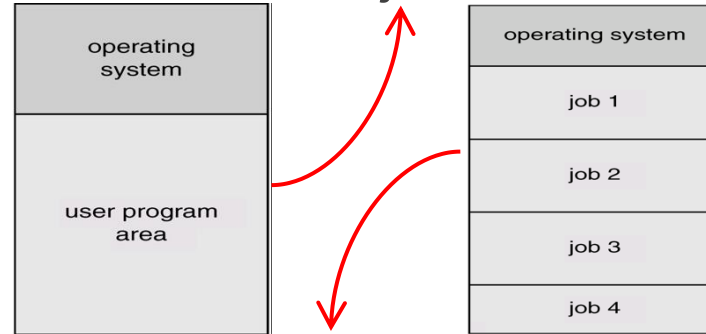
- **Interfaccia** - intermediario tra un utente e l'hardware di un computer.
- **Allocatore di Risorse** – gestisce ed alloca in modo efficiente le risorse.
- **Programma di Controllo** – controllo l'esecuzione dei programmi utente e le operazioni dei dispositivi di I/O.
- **Kernel** – l'unico programma che è sempre in esecuzione (tutti gli altri sono considerati programmi applicativi).

- **Obiettivi di un SO:**
 - Eseguire i programmi utente
 - Facilitare la soluzione dei problemi degli utenti.
 - Rendere il computer conveniente.

Classificazione dei Sistemi Operativi

■ Batch

- Un solo job + s.o. in memoria
- CPU dedicata al job



■ Multitasking

- Diversi job + s.o. in memoria
- CPU multiplata tra loro

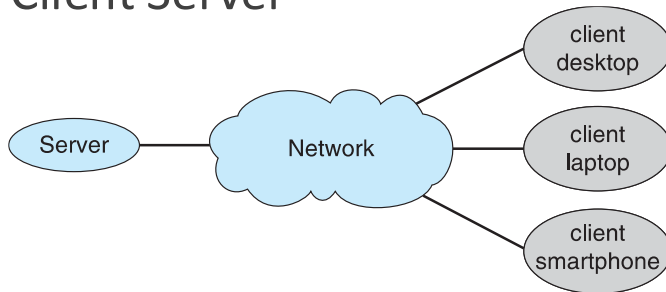
- > *multiprogramming* - un job cede volontariamente la CPU quando non gli serve (es. usa I/O)
- > *time-sharing* - il s.o. può togliere la CPU anche ad un job che ancora la sta utilizzando
- > *real-time* - esistono vincoli temporali precisi
 - soft real time - applicazioni multimediali, virtual reality, ...
 - hard real time - applicazioni industriali, robotica, ...

Classificazione dei Sistemi Operativi

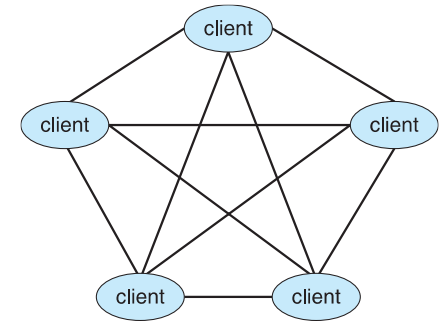


Classificazione dei Sistemi Operativi

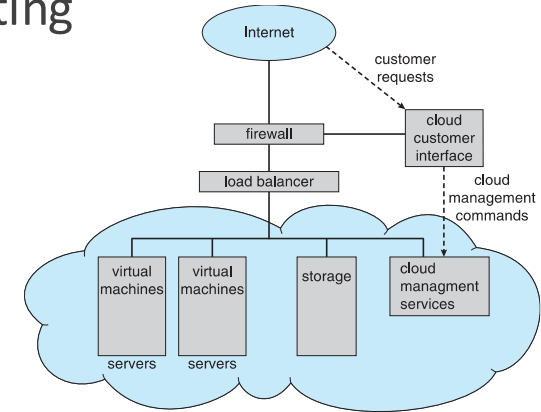
- Traditional
 - Macchine di tipo stand-alone general-purpose
 - Interconnected via wired or wireless networks
- Mobile
 - Smartphones, tablets, etc.
 - Interconnected via wireless networks
 - Caratteristiche extra: OS (GPS, giroscopio), App. (realtà aumentata)
- Client Server



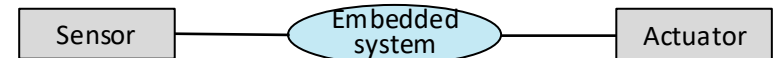
- Pear-to-Pear



- Cloud computing



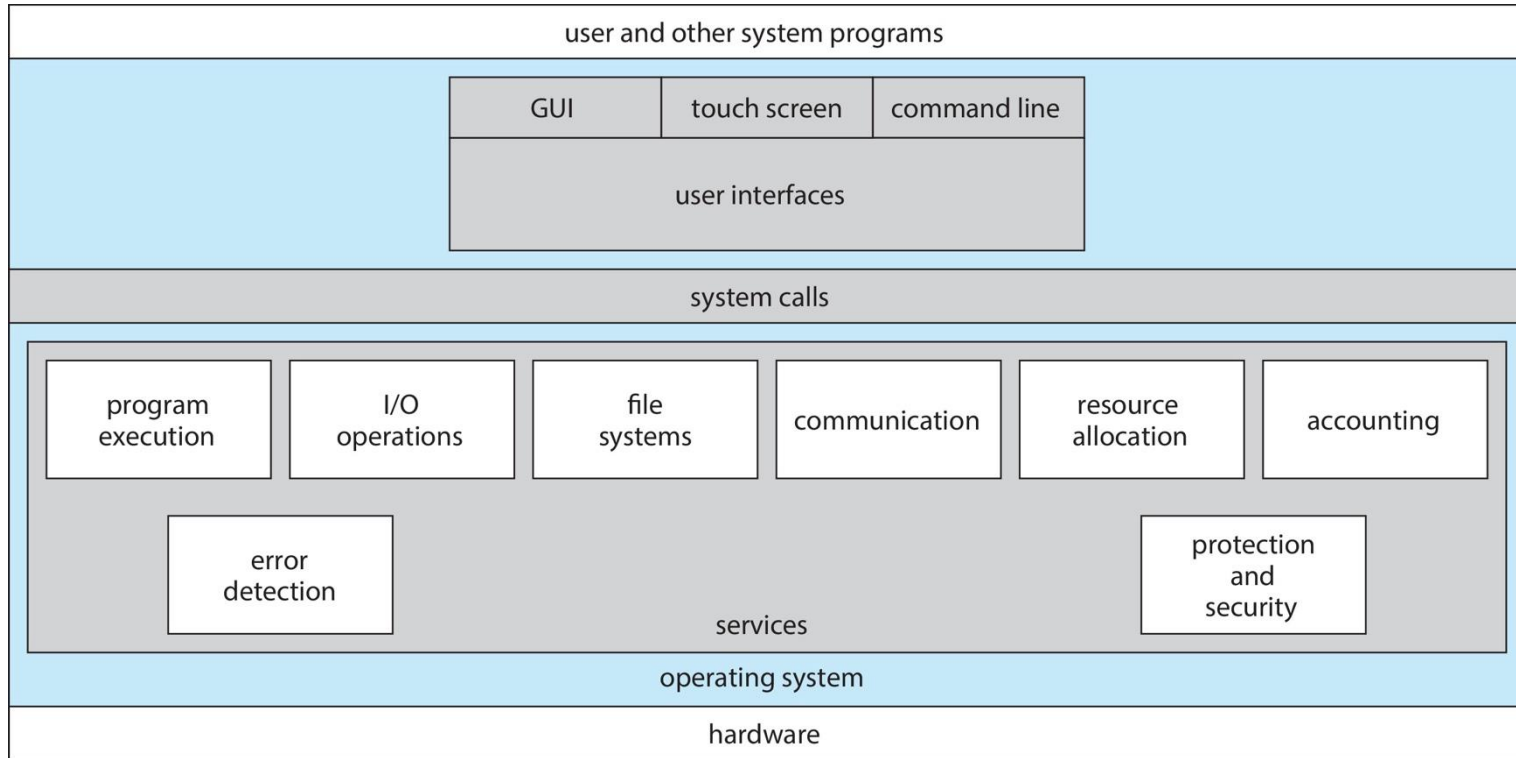
- Real-time Embedded



- ✓ Sistemi di Elaborazione
 - ✓ Classificazione basata sulle architetture hardware (di Flynn)
 - ✓ Classificazione basata sui sistemi operativi
- Struttura di un Sistema Operativo
 - Funzionalità di un SO
 - Servizi di un SO
 - Architetture
 - Monolitica
 - Stratificata
 - Microkernel
 - Modulare
 - Macchina Virtuale

- **Struttura in base alla funzione**
 - Gestione dei processi
 - Gestione della memoria centrale
 - Gestione dei dispositivi di I/O
 - Gestione della memoria secondaria
 - Gestione del file system
 - Gestione delle reti
 - Gestione della Sicurezza
 - Interprete di comandi
- **Struttura in base al livello**
 - Servizi di sistema
 - Programmi di sistema

Struttura dei Sistemi Operativi



Funzione: Gestione dei Processi

- Un **processo** è un **programma in esecuzione**
- Ogni processo ha bisogno di determinate risorse per terminare il suo compito: CPU, memoria, file e dispositivi di I/O.
- Il SO è responsabile della gestione dei processi:
 - Creazione e cancellazione dei processi,
 - Sospensione e ripristino dei processi,
 - Meccanismi per :
 - la sincronizzazione dei processi
 - la comunicazione dei processi

Funzione: Gestione della Memoria

- La **memoria** centrale è un **array di word o byte** molto grande, ognuno con un suo indirizzo. Permette l'accesso veloce a dati condivisi dalla CPU e dai dispositivi di I/O.
- La memoria centrale è un dispositivo di memorizzazione volatile.
- Il SO è responsabile delle seguenti attività relative alla gestione della memoria:
 - Tenere traccia di quali parti della memoria sono utilizzate e da chi.
 - Decidere quali processi caricare in memoria.
 - Allocare e deallocare lo spazio di memoria.

Funzione: Gestione dei File

- Un **file** è una collezione di informazioni collegate tra loro e definite dal suo creatore. Di solito un file rappresenta o un programma (sorgente o oggetto) o dei dati.
- Il SO è responsabile delle seguenti attività relative alla gestione dei file:
 - Creazione e cancellazione dei file.
 - Creazione e cancellazione di directory (cartelle).
 - Mettere a disposizione primitive per la manipolazione di file e directory.
 - “Mappare” i file su dispositivi di memorizzazione secondaria e/o altri dispositivi di I/O.
 - Gestire il backup dei file.

Funzione: Gestione dell' I/O

- Il sistema di gestione dell' I/O consiste in:
 - Un sistema di buffer-caching
 - Una interfaccia generale per i driver dei dispositivi
 - Driver per specifici dispositivi hardware

Funzione: Gestione della Memoria Secondaria

- La maggior parte dei moderni computer utilizza i dischi come principale mezzo di memorizzazione sia per programmi sia per i dati.
- Il SO è responsabile delle seguenti attività relative alla gestione dei dischi:
 - Gestione dello spazio libero
 - Allocazione dello spazio
 - Scheduling del disco

Funzione: Gestione delle Reti

- Un sistema distribuito è una collezione di processori che non condividono né la memoria né il clock.
- Il SO è responsabile delle seguenti attività relative alla gestione delle reti:
 - Invio e ricezione di messaggi
 - Instradamento e gestione delle connessioni
 - Gestione dei conflitti
 - Gestione della sicurezza

- La protezione si riferisce al meccanismo per controllare l'accesso di processi e utenti alle risorse di un utente e a quelle di sistema.
- Il sistema di protezione deve:
 - distinguere tra un utilizzo autorizzato ed uno non autorizzato
 - specificare quali controlli devono essere eseguiti ed i mezzi per effettuarli

Funzione: Interprete dei Comandi

Command Line Interpreter (CLI)

- Gran parte dei comandi sono dati al SO per mezzo di istruzioni. Tali istruzioni permettono:
 - la creazione e la gestione dei processi
 - la gestione dell' I/O
 - la gestione della memoria secondaria
 - secondary-storage management
 - la gestione della memoria centrale
 - l' accesso al file system
 - la protezione delle risorse
 - il networking
- Il programma che legge ed interpreta tali istruzioni è chiamato in diversi modi:
 - interprete comandi
 - shell (in UNIX)
 - prompt di MS-DOS (in MS Windows)

Esistono anche interfacce utente grafiche

- Finder (MacOS)
- Explorer (MS Windows)
- X Window System (Unix)

Livello: Servizi di un SO

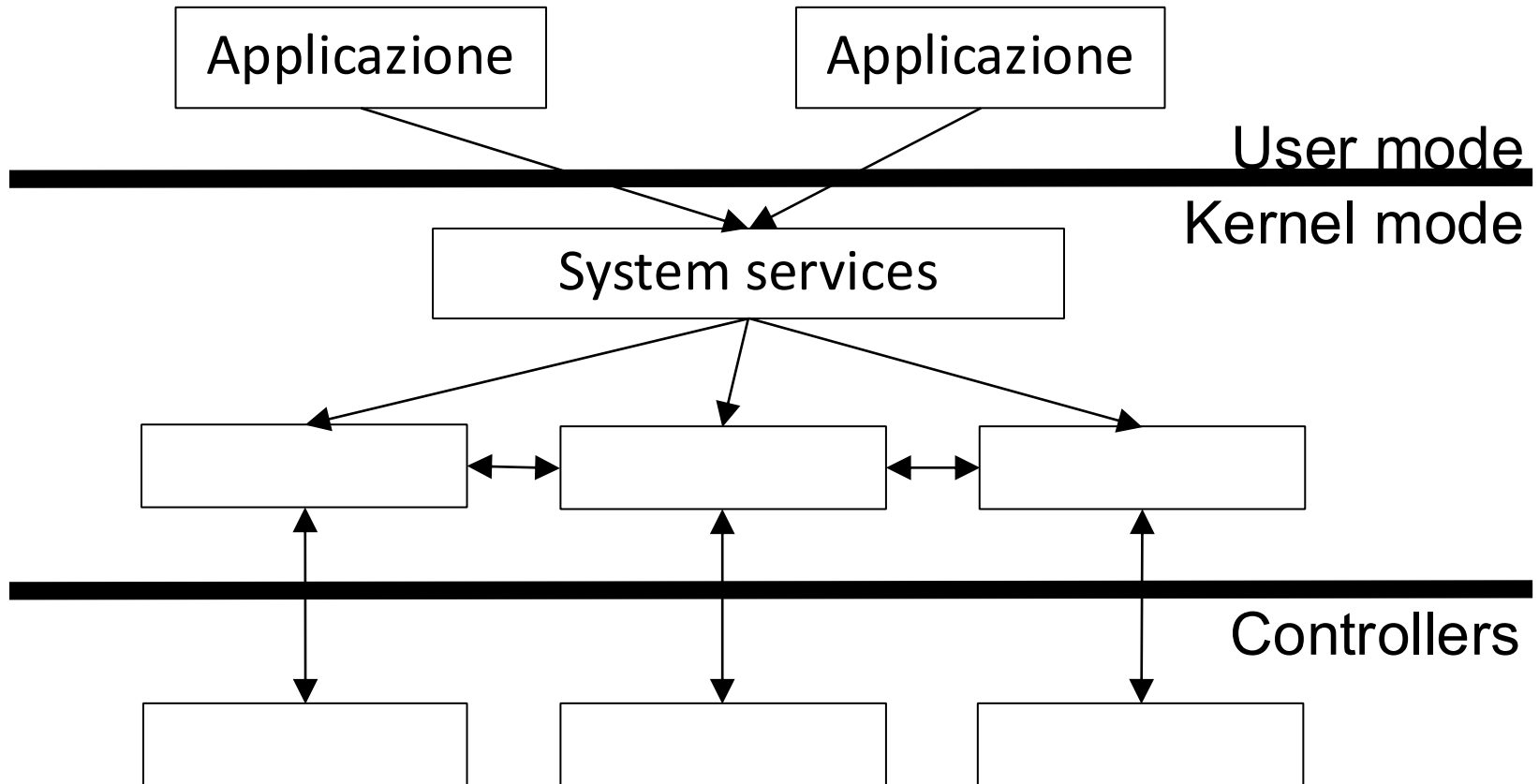
- Caricamento ed esecuzione dei programmi.
- Operazioni di I/O – un utente non può eseguirle direttamente.
- Manipolazione del File-system – la possibilità per un processo o un utente di leggere, scrivere, creare e cancellare file.
- Comunicazione – la possibilità per due processi (sulla stessa macchina o su due macchine differenti collegate tra loro in rete) di scambiarsi informazioni.
- Rilevamento di errori.
- Allocazione delle risorse – a più utenti/processi in esecuzione nello stesso momento.
- Contabilizzazione dell'uso delle risorse – tenere traccia di chi e per quanto tempo ha utilizzato che cosa. Questo serve sia per raccogliere statistiche sul funzionamento del sistema sia nei sistemi in cui l'accesso alle risorse è a pagamento.
- Protezione – assicurare che l'accesso alle risorse del sistema sia controllato.

Livello: Programmi di sistema

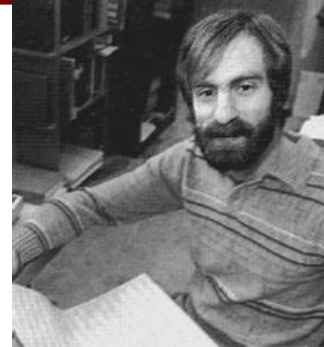
- Costituiscono l'ambiente di sviluppo ed esecuzione dei programmi:
 - Manipolazione e modifica file
 - Stato del sistema
 - Strumenti di ausilio alla programmazione
 - Caricamento ed esecuzione di programmi
 - Comunicazione
 - Programmi di utilità
- Per la maggior parte degli utenti, il SO è caratterizzato dai programmi di sistema che mette a disposizione piuttosto che dalle system call.

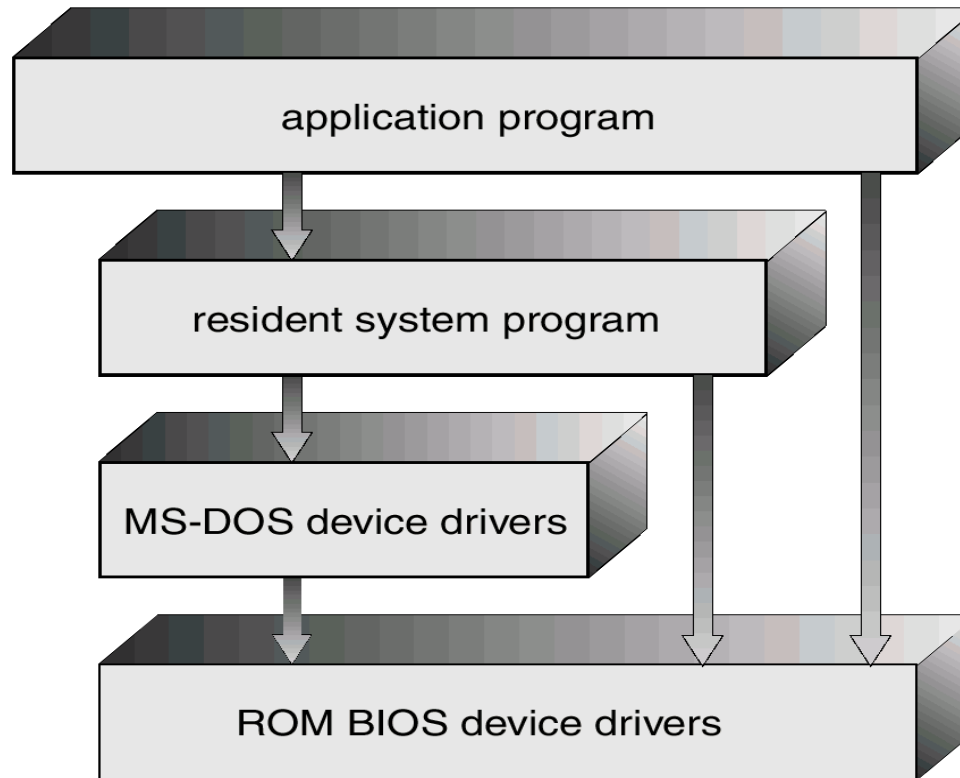
- ✓ Sistemi di Elaborazione
 - ✓ Classificazione basata sulle architetture hardware (di Flynn)
 - ✓ Classificazione basata sui sistemi operativi
- Struttura di un Sistema Operativo
 - ✓ Funzionalità di un SO
 - ✓ Servizi di un SO
 - Architetture
 - Monolitica
 - Stratificata
 - Microkernel
 - Modulare
 - Macchina Virtuale

Architettura Monolitica



- MS-DOS è stato scritto (da **Tim Paterson**) tenendo conto delle limitazioni dell' hardware e per dare il massimo delle funzionalità occupando il minimo spazio. Infatti MS-DOS, anche se ha un minimo di strutturazione:
 - non è diviso in moduli
 - le interfacce e i livelli di funzionalità non sono ben separati



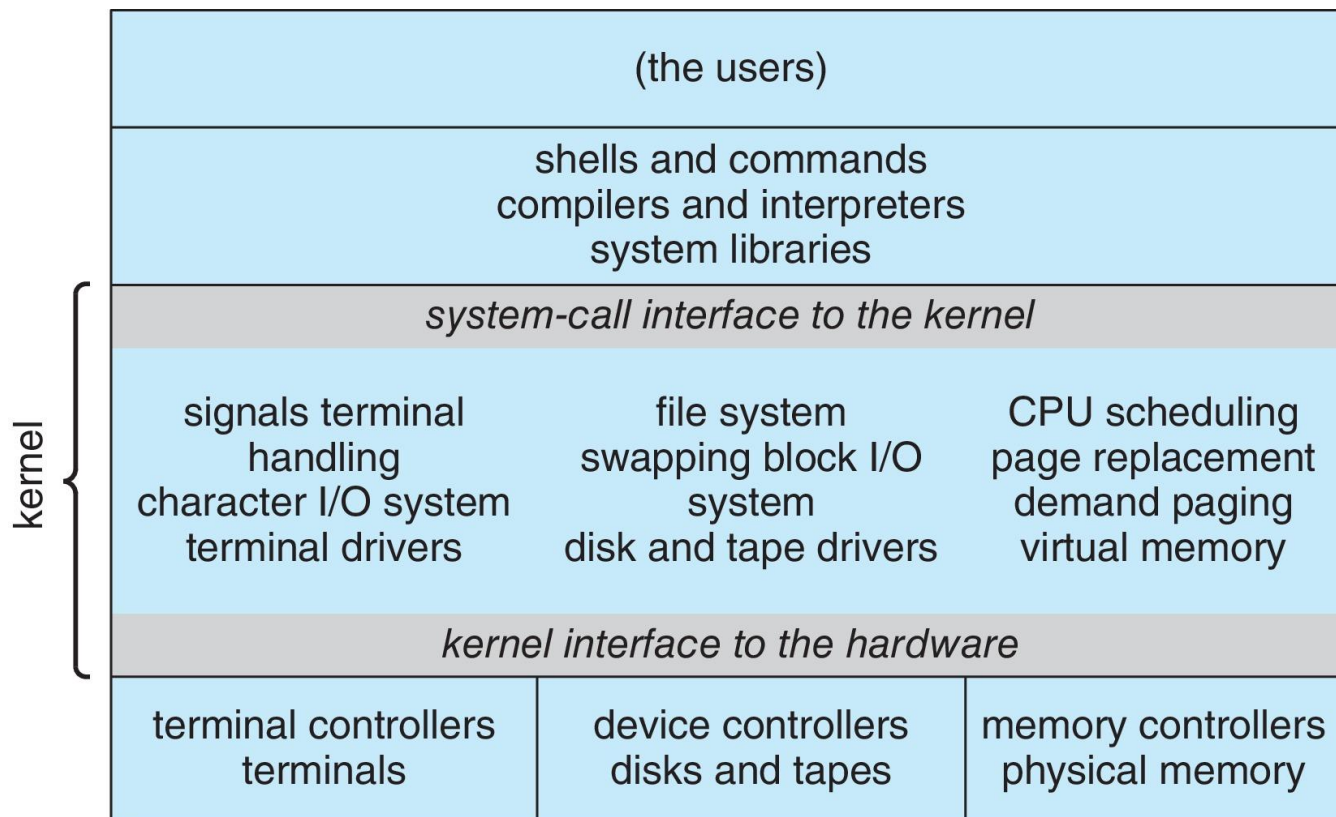


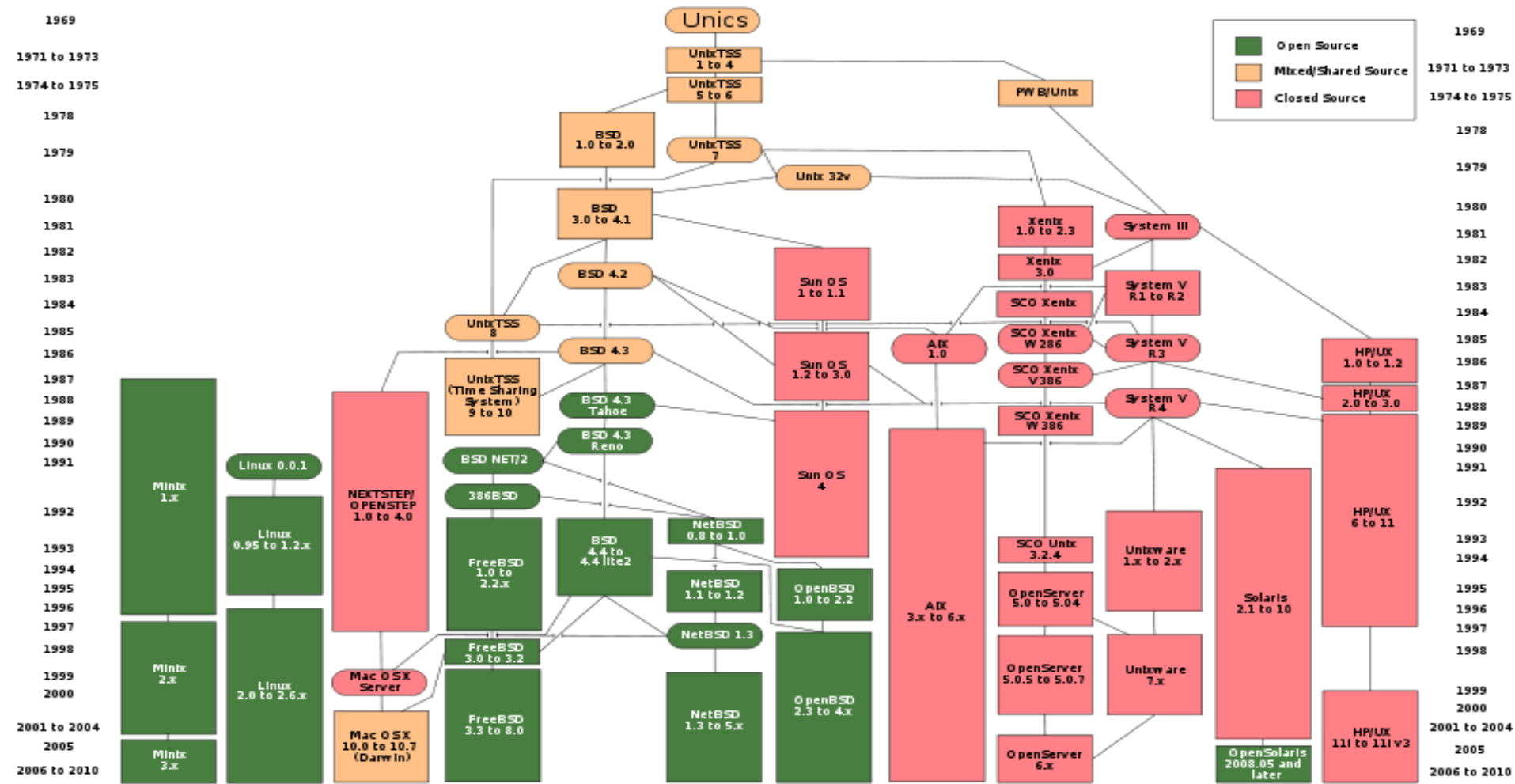
- UNIX – è stato scritto (da **Ken Thompson** e **Dennis Ritchie**) tenendo conto delle limitazioni dell'hardware.



Quindi lo UNIX originale ha una strutturazione limitata. Il sistema consiste di due parti:

- programmi di sistema
- il kernel
 - tutto ciò che è compreso tra l'interfaccia con le system call e l'hardware del sistema
 - tutte le funzioni di gestione della CU, della memoria ed altre ancora sono riunite in un unico livello.

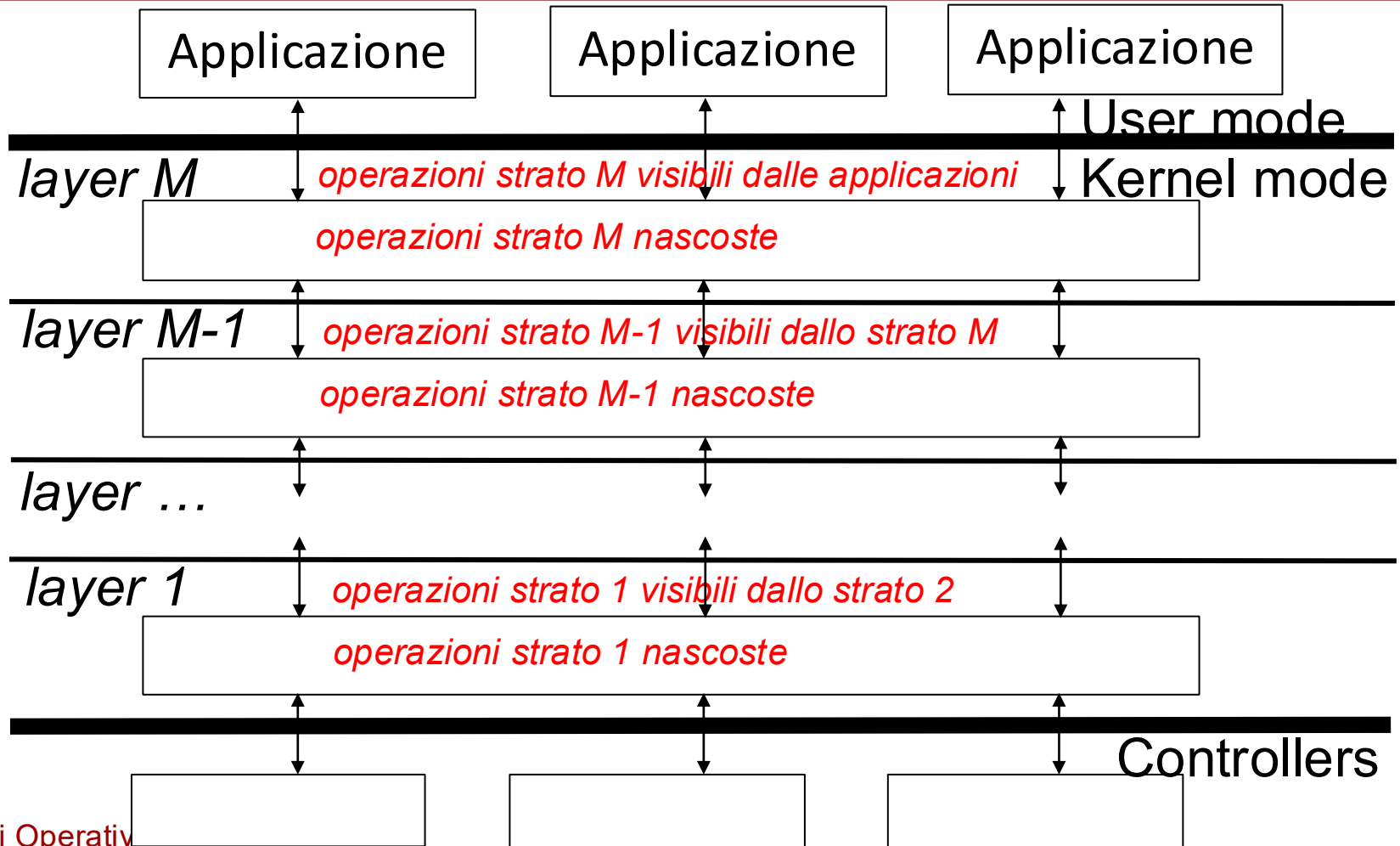


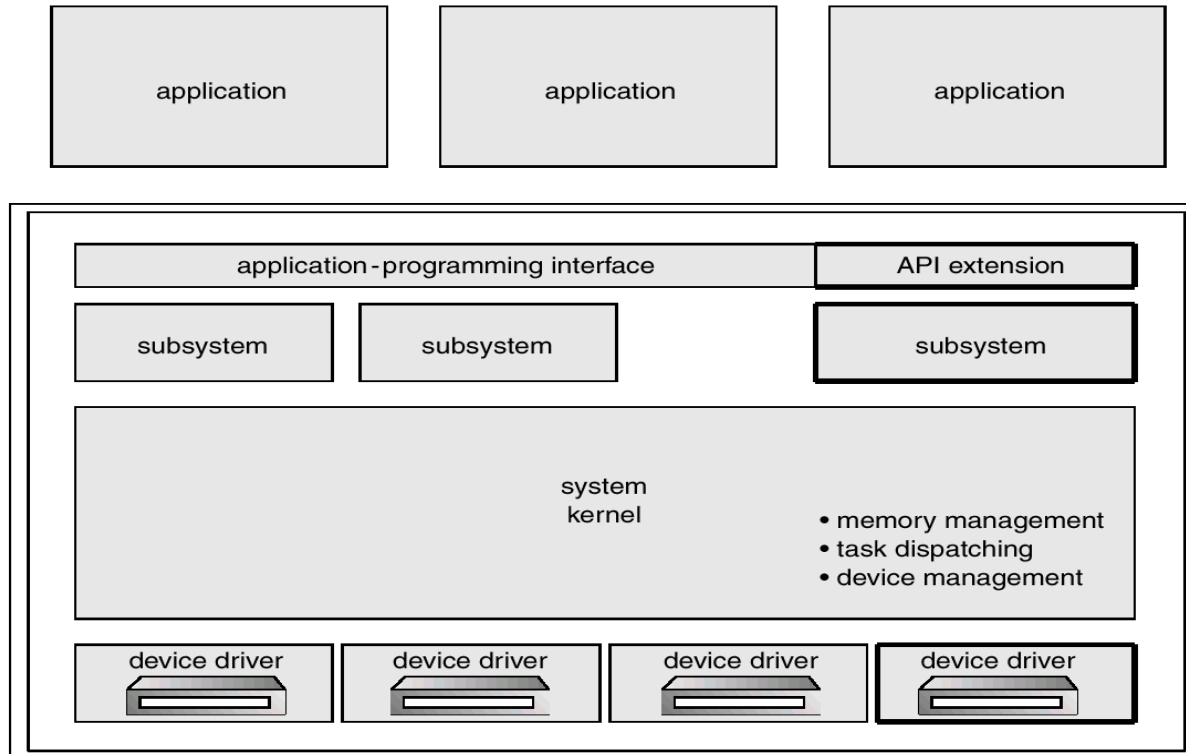


Architettura Stratificata

- Il SO è suddiviso in un certo numero di strati (chiamati **livelli**).
- Ogni strato è costruito sopra gli strati precedenti. Il livello più basso (livello 0) è l'hardware ed il livello più alto (livello N) è l'interfaccia utente.
- Per rendere il sistema modulare, i livelli sono definiti in modo tale che ogni livello utilizza soltanto funzioni (operazioni) e servizi che appartengono ai livelli inferiori.

Architettura Stratificata

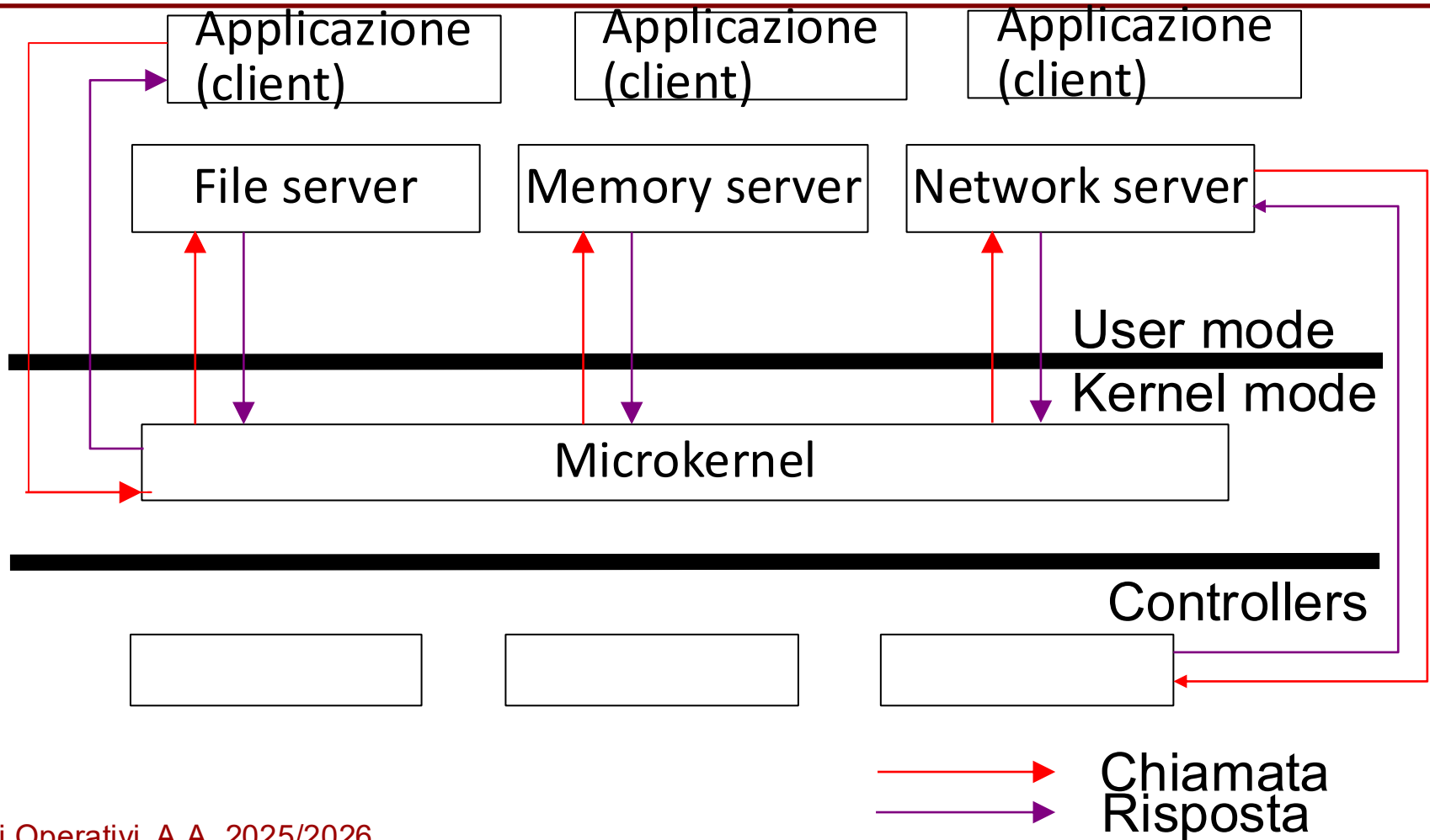




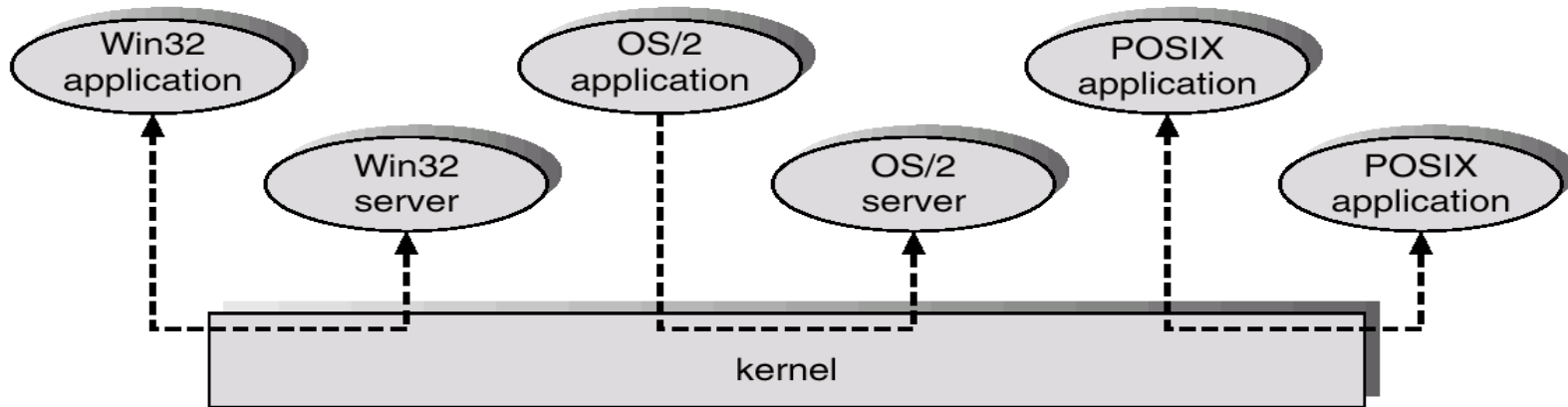
Architettura a Microkernel (Client/Server)

- Spostare tutto quanto è possibile dal kernel allo spazio “utente”.
- La comunicazione tra i moduli utente avviene utilizzando lo scambio di messaggi.
- Benefici:
 - è più facile estendere un microkernel
 - è più facile fare il porting del SO su nuove architetture
 - si adatta più facilmente ai sistemi distribuiti
 - più affidabile (meno codice gira in modalità kernel)
 - più sicuro

Architettura a Microkernel (Client/Server)



Windows NT



- Le architetture a microkernel sono state ritenute da Linus Torvalds e da Ken Thompson inutilmente troppo complicate (memorabile la disputa con Andrew Tanenbaum nel 1992).
- È stata proposta da Linus Torvalds una architettura a metà strada tra l'architettura monolitica e quella stratificata.

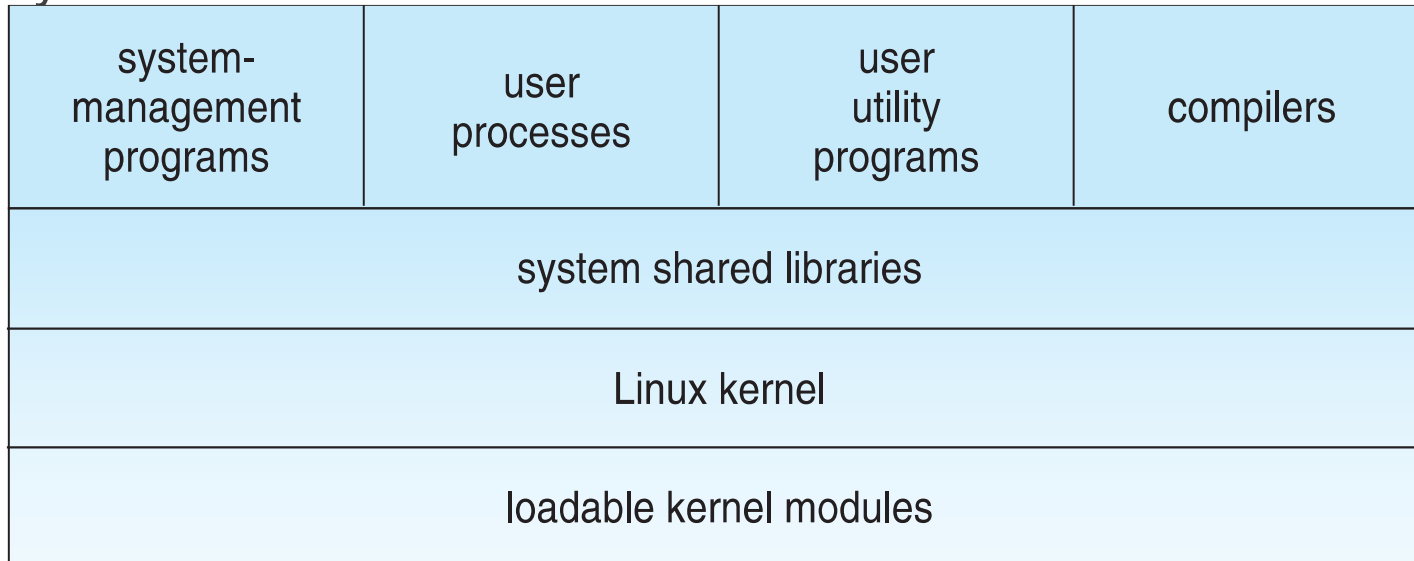
Linux (scritto da **Linus Torvalds**)

adotta una **architettura ibrida**

- adotta una architettura monolitica
- adotta i moduli
 - così non si deve ricompilare il kernel ogni volta che si aggiunge un driver o una nuova funzionalità
 - basta caricare il modulo e *linkarlo* dinamicamente per aggiungere una nuova funzionalità
 - il modulo è software che gira in modalità kernel, se contiene degli errori può danneggiare il kernel (il sistema si può bloccare)



- Come in gran parte delle implementazioni di Unix, Linux ha tre componenti principali:
 - Kernel
 - System libraries
 - System utilities

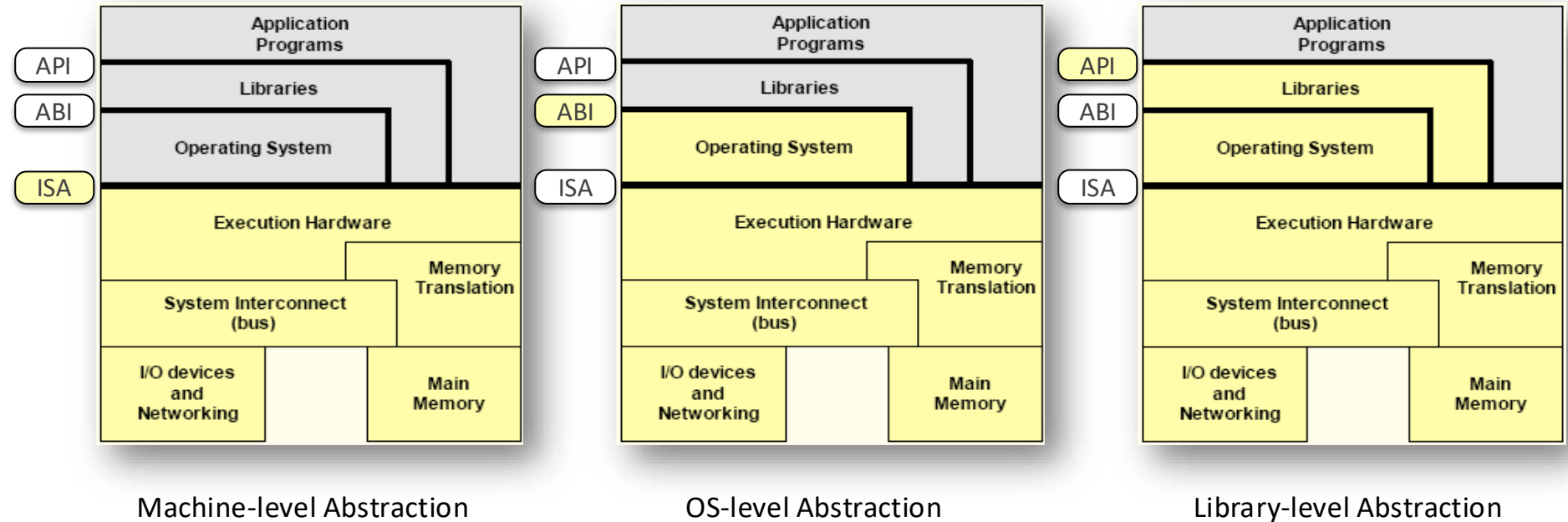


- **Il kernel**
 - Realizza tutte le funzioni del sistema operativo e serve per isolare il resto del sistema dalle caratteristiche dell'hardware
 - Viene eseguito in *kernel mode* ed ha accesso a tutte le risorse fisiche del sistema
 - Tutto il codice del kernel e le strutture dati sono mantenuti nello stesso spazio degli indirizzi
- **Le librerie di sistema**
 - Definiscono un insieme standard di funzioni attraverso le quali le applicazioni interagiscono con il kernel
 - Vengono eseguite in *user mode*
- **Le system utilities**
 - Eseguono dei compiti di gestione specializzati
 - Vengono eseguite in *user mode*

■ | moduli del kernel

- Sono sezioni del kernel che possono essere compilate, caricate e scaricate in modo indipendente dal resto del kernel.
- Un modulo del kernel tipicamente serve per realizzare un driver di un dispositivo, un file system o un protocollo di rete.
- Questo permette a terze parti di distribuire driver o file system che non possono essere distribuiti con licenza GPL.
- Questo permette a linux di essere configurato con un kernel minimo, senza moduli esterni.

Livelli di Astrazione

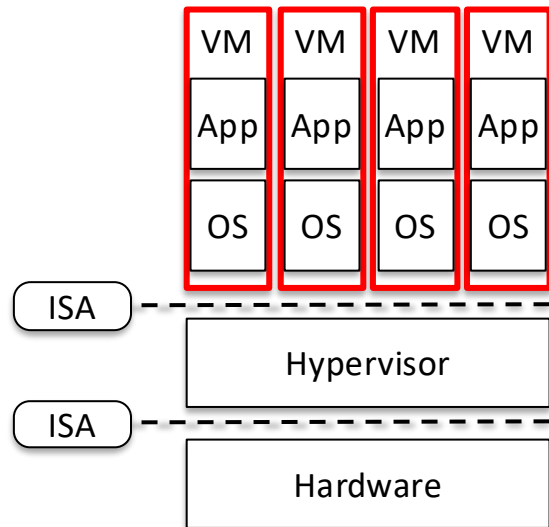
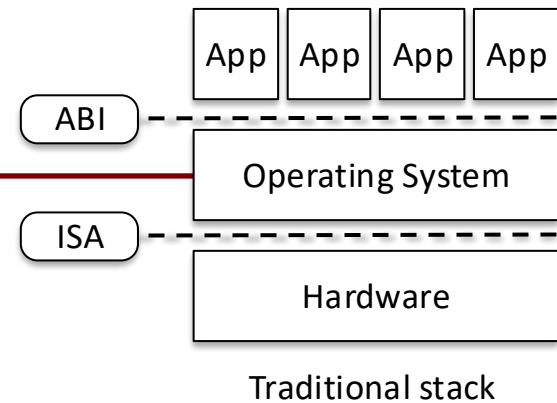


Legenda

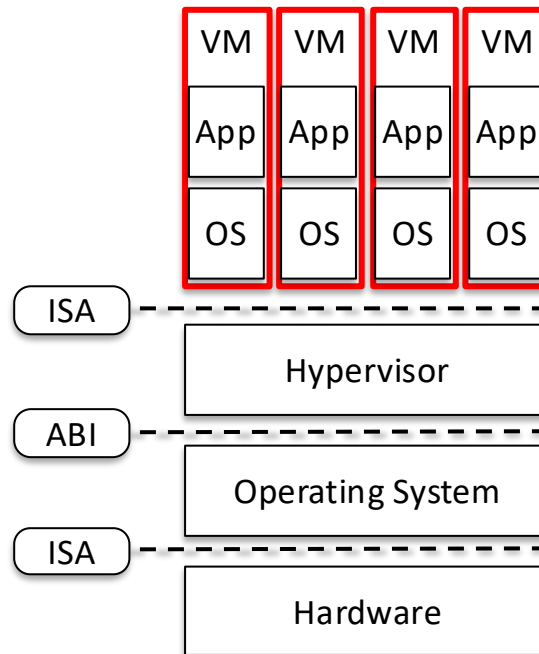
- ISA – Instruction Set Architecture
- ABI – Application Binary Interface
- API – Application Programming Interface

- La virtualizzazione è una tecnica che consente di creare una versione astratta di una risorsa hardware o software, permettendo a più sistemi operativi o applicazioni di girare simultaneamente sullo stesso hardware fisico. Un **hypervisor** (o Virtual Machine Monitor) gestisce e isola le macchine virtuali, assegnando loro risorse come CPU, RAM e storage.
- La chiave è che l'hardware della macchina ospitante (host) e quello della macchina virtuale (guest) sono **della stessa architettura**. Il software guest gira quindi quasi direttamente sull'hardware reale, con un overhead minimo.
 - Esempi: VMware, VirtualBox, KVM, UTM, Hyper-V, Xen.

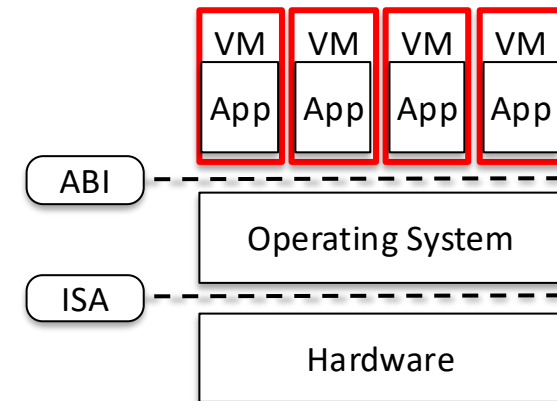
Virtualizzazione



a) Hardware-level virtualization
Type I Hypervisor – Bare-metal (Es. WSL2)



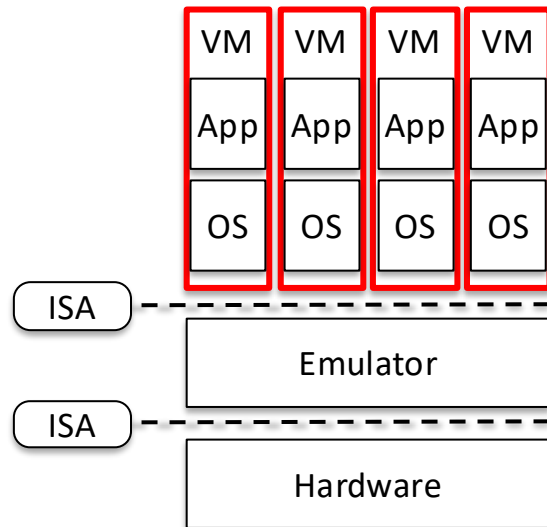
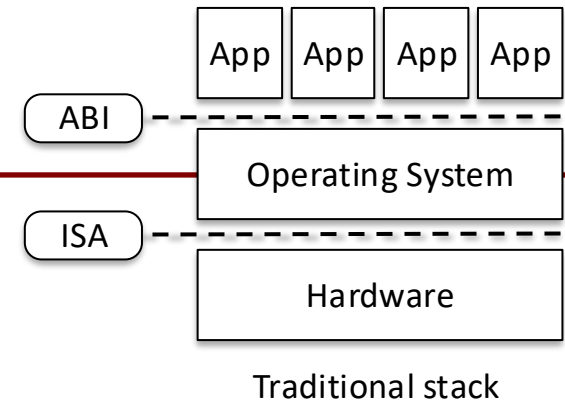
b) Hardware-level virtualization
Type II Hypervisor – Hosted (Es. VirtualBox)



c) Operating System-level virtualization
Container-based (Es. Docker)

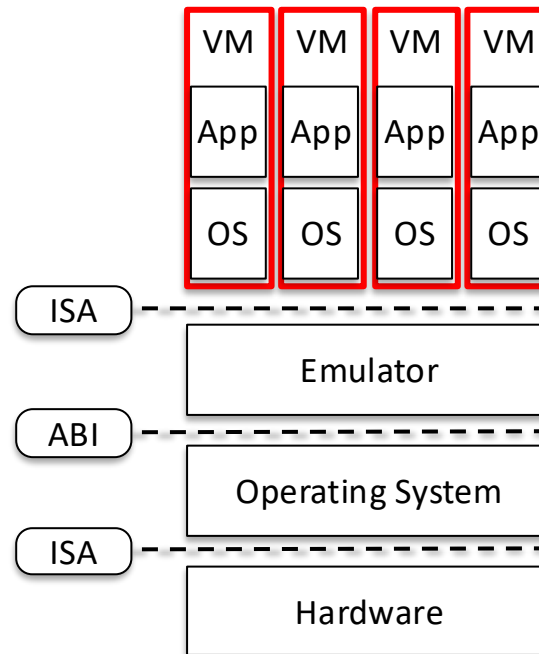
- L'emulazione consiste nel riprodurre il comportamento di un sistema diverso — hardware o software — attraverso un programma (l'emulatore).
- Emulatore (in senso stretto) – emulazione dell'hardware
 - Traduce le istruzioni dell'architettura emulata in istruzioni comprensibili dall'architettura reale.
 - Esempi: QEMU (in modalità emulazione), emulatori di console di videogiochi
- Compatibility layer – emulazione (in senso lato) del kernel
 - Traduce le system call al sistema operativo guest in system call al sistema operativo host
 - Esempi: compatibility layer come Wine, WSL1, e QEMU (in modalità userspace)

Emulazione



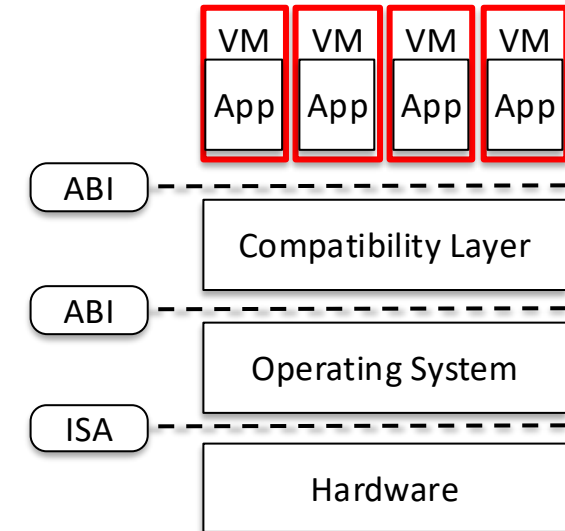
a) Hardware-level emulation

Type I Hypervisor – Bare-metal (Es. RetroArch)



b) Hardware-level emulation

Type II Hypervisor – Hosted (Es. QEMU)



c) Operating System

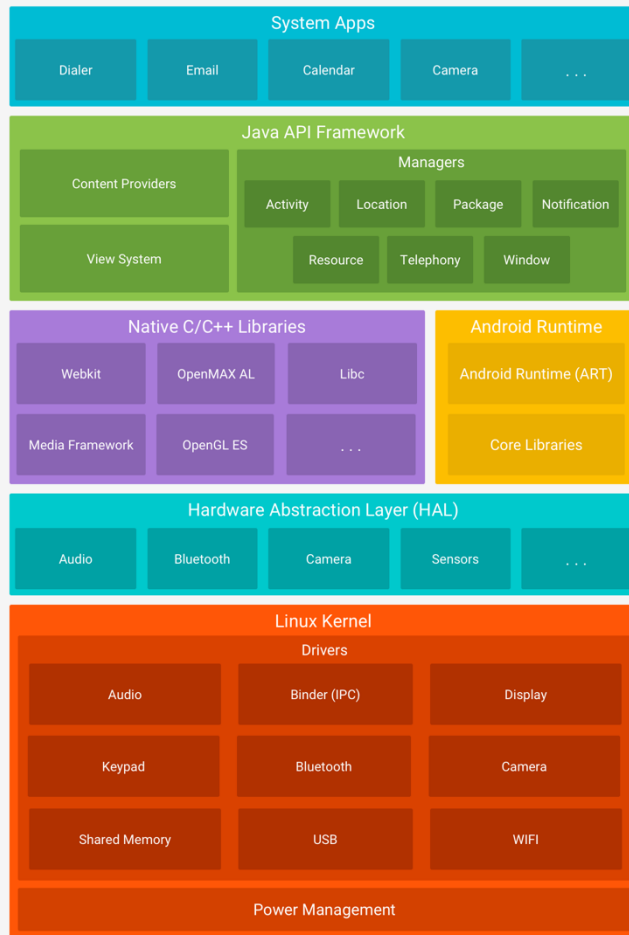
compatibility layerd (Es. WSL1)

Architettura a Macchine Virtuali

Vantaggi e Svantaggi

- Il concetto di macchina virtuale fornisce una completa protezione alle risorse del sistema, infatti ogni macchina virtuale è isolata dalle altre. Questo isolamento non permette però di condividere le risorse (si pensi al disco).
- Una macchina virtuale è il veicolo ideale per la ricerca e lo sviluppo di nuovi SO. Infatti lo sviluppo viene fatto sulla macchina virtuale e questo non danneggia il normale funzionamento della macchina.
- Una macchina virtuale è difficile da realizzare a causa della difficoltà nel fornire un esatto duplicato della macchina sottostante.
- Il concetto di macchina virtuale è ritornato di moda recentemente per risolvere i problemi di interoperabilità
 - cloud computing
 - emulatori di un S.O. per macchine diverse (es. Emulatore Linux per Windows, ecc.)
 - la macchina virtuale di Java

Architettura di Android



Type II hypervisor – esegue codice scritto in Dalvik bytecode (DEX – Dalvik Executable)

SELinux – netta separazione tra processi